

AL Objects Cheat Sheet

Business Central development · Talan Academy

For AL developers and technical consultants

WHAT'S INSIDE

- AL basics, the edit-compile-publish loop, and the object map
- 13 AL object types, each with a minimal example
- Properties, triggers & extension-model checklist

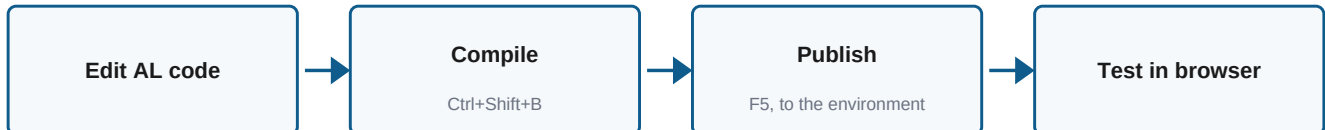
00 AL basics & the object map

What AL is — The language Business Central extensions are written in. It compiles to a single portable **.app** package that installs into an environment without ever touching the base application's own source.

The toolchain — VS Code plus the **AL Language** extension. **app.json** is the manifest (id, name, version, dependencies, idRanges); **launch.json** tells VS Code which environment to compile and publish against.

Symbols — Before you can reference a base-app table or page, VS Code downloads its **symbols** (Ctrl+Shift+P → "Download Symbols") — a read-only signature of every standard object, refreshed whenever the target environment changes version.

THE EDIT-COMPILE-PUBLISH LOOP



THE OBJECT MAP — THREE LAYERS

Every AL object belongs to one of three layers. Knowing which one answers “where does this logic go” before you’ve even opened VS Code.

Data layer	UI layer	Logic layer
• Table • Table extension • Enum • Enum extension • Query Stores and structures business data.	• Page • Page extension Presents and manipulates data.	• Codeunit • Report • Report extension • XMLport • Profile • Permission set Implements business rules and everything else that isn't pure storage or pure screen.

01 Table

What it is — Primary database object: structured business data behind a unique primary key.

When to use — Persistent data that will be queried, updated, and reported on over time.

```
table 50100 "Tal Rental Equipment"
{
    Caption = 'Rental Equipment';
    DataClassification = CustomerContent;
    fields
    {
        field(1; "No."; Code[20]) { }
        field(2; Description; Text[100]) { }
    }
    keys { key(PK; "No.") { Clustered = true; } }
}
```

TIP

Assign field IDs from your reserved range, and set a Clustered primary key.

03 Enum

What it is — A closed, predefined list of values, each with a numeric ID and a caption.

When to use — A field's valid values form a fixed, known set (status, type, category).

```
enum 50100 "Tal Equipment Type"
{
    Extensible = true;
    value(0; Projector) { Caption = 'Projector'; }
    value(1; Laptop) { Caption = 'Laptop'; }
}
```

TIP

Mark Extensible = true unless you deliberately want to block future values.

05 Page

What it is — The UI object that displays and edits table data — List, Card, FactBox, and more.

When to use — A user needs to view or edit records. Never edit a table directly.

```
page 50100 "Tal Rental Equipment List"
{
    PageType = List;
    SourceTable = "Tal Rental Equipment";
    layout
    {
        area(Content)
        {
            repeater(General)
            {
                field("No."; Rec."No.") { }
            }
        }
    }
}
```

TIP

Pick the right PageType deliberately — it isn't cosmetic, it changes behavior.

02 Table extension

What it is — Adds fields or keys to an existing table without modifying the original object.

When to use — Data belongs conceptually to a standard table (Item, Customer) with no home of its own.

```
tableextension 50101 "Tal Item Rental"
extends Item
{
    fields
    {
        field(50100; "Is Rental Equipment";
            Boolean) { }
    }
}
```

TIP

Field IDs must come from your own range — never reuse a standard field's ID.

04 Enum extension

What it is — Adds values to an existing enum without touching the original object.

When to use — A standard or third-party enum is missing a value your business genuinely needs.

```
enumextension 50101 "Tal Equipment Type Ext"
extends "Tal Equipment Type"
{
    value(50100; Camera) { Caption = 'Camera'; }
}
```

TIP

Value IDs must come from your own range — a collision blocks installation.

06 Page extension

What it is — Adds fields, groups, actions, or FactBoxes to an existing page.

When to use — Surface a new field (often from a table extension) on a standard screen.

```
pageextension 50101 "Tal Item Card Rental"
extends "Item Card"
{
    layout
    {
        addafter(Type)
        {
            field("Is Rental Equipment";
                Rec."Is Rental Equipment") { }
        }
    }
}
```

TIP

Use addafter()/addlast() deliberately — placement affects usability.

07 Codeunit

What it is — A reusable container for business-logic procedures, with no UI of its own.

When to use — Logic reused across more than one page/report, or any multi-step process.

```
codeunit 50100 "Tal Rental Management"
{
    procedure Available(No: Code[20]): Boolean
    var
        Equip: Record "Tal Rental Equipment";
    begin
        exit(Equip.Get(No) and Equip.Available);
    end;
}
```

TIP

Keep the public surface small — expose only what other objects truly need.

09 Report extension

What it is — Adds columns or controls to an existing standard report.

When to use — A standard report is 90% of what you need, plus a handful of columns.

```
reportextension 50101 "Tal Customer List Rental"
    extends "Customer - List"
{
    dataset
    {
        add(Customer)
        {
            column(IsRentalCustomer;
                "Is Rental Customer") { }
        }
    }
}
```

TIP

Use addlast() on the dataset — the goal is additive, not a redesign.

11 XMLport

What it is — Moves data between Business Central and an external file (XML or delimited text).

When to use — Import a catalogue or price list, or export documents to another system.

```
xmlport 50100 "Tal Import Rental Equipment"
{
    Direction = Import;
    Format = VariableText;
    schema
    {
        tableelement(Equip; "Tal Rental Equipment")
        {
            fieldelement(No_; Equip."No.") { }
        }
    }
}
```

TIP

Validate and reject bad rows in OnBeforeInsertRecord — don't trust the file.

08 Report

What it is — A document object that summarizes data for printing or export, with its own layout.

When to use — A formatted, often printable or exportable view of aggregated or filtered data.

```
report 50100 "Tal Equipment Availability"
{
    UsageCategory = ReportsAndAnalysis;
    ApplicationArea = All;
    dataset
    {
        dataitem(Equip; "Tal Rental Equipment")
        {
            column(No_; "No.") { }
        }
    }
}
```

TIP

Source aggregation from a query, not a hand-rolled loop, when it spans records.

10 Query

What it is — A read-only object combining tables with grouping, filtering, and aggregation.

When to use — Aggregated views, KPIs, or analytics — not a single unaggregated record.

```
query 50100 "Tal Rental Equipment Summary"
{
    QueryType = Normal;
    elements
    {
        dataitem(Equip; "Tal Rental Equipment")
        {
            column(EquipType; "Equipment Type") { }
            column(ItemCount; "No.") { Method = Count; }
        }
    }
}
```

TIP

Every dataitem you add is a join the query has to execute — keep it lean.

12 Profile

What it is — Defines a user role: names it, points at a Role Center, carries page customizations.

When to use — A group of users shares a job and needs a tailored workspace.

```
profile "Tal Rental Manager"
{
    Caption = 'Rental Manager';
    RoleCenter = "Tal Rental Manager RC";
    Enabled = true;
}
```

TIP

A profile is identified by name, not a number — the one exception in AL.

13 Permission set

What it is — Lists exactly what a user may read, insert, modify, delete, or execute per object.

When to use — Every extension needs at least one — BC blocks unpermitted objects by default.

```
permissionset 50100 "Tal Rental Full"
{
    Assignable = true;
    Permissions =
        tabledata "Tal Rental Equipment" = RIMD,
        codeunit "Tal Rental Management" = X;
}
```

TIP

Ship a full-access and a read-only set as a pair, named with your prefix.

14 Properties & triggers quick reference

The properties and triggers every AL object leans on, regardless of type — the ones worth knowing cold before reaching for the documentation.

ESSENTIAL PROPERTIES

Property	Applies to	Notes
DataClassification	Table, table field	Required on every table/field. Use CustomerContent for ordinary business data; never leave it at ToBeClassified.
ApplicationArea	Page field/action, Report	Almost always All on an internal app — it hides the control otherwise, silently, with no error.
Access	Any object	Public (default) or Internal. Internal hides the object from other extensions — use it for anything not meant as a public API.
ObsoleteState	Any object or field	No / Pending / Removed. Mark Pending with a reason before deleting — never delete outright while anyone may still reference it.
SourceTable	Page, Report	The record the object is bound to. A page with no SourceTable can only host actions and static content.
SourceTableView	Page	A default filter and sort baked into the page itself, independent of whatever the caller passes in.
Editable	Page, page field	Set false on a page (or one field) that is read-only — a List page summarizing posted documents, for example.
Extensible	Enum	true unless you deliberately want to block every future extension value.

COMMON TRIGGERS BY OBJECT

Table — OnInsert, OnModify, OnDelete, OnRename, field-level OnValidate

Page — OnOpenPage, OnAfterGetRecord, OnInsertRecord, OnModifyRecord, OnDeleteRecord, field/action-level OnValidate / OnAction

Codeunit — OnRun — the single entry point when the codeunit is called as a task, not just a procedure library

Report / Query dataitem — OnPreDataItem, OnAfterGetRecord, OnPostDataItem

PITFALL

ApplicationArea = All is the single most common reason a field or action "isn't showing up" for a tester — check it before anything more exotic.

15

Naming, ID range & extension-model checklist

- ☐ Decide a short project prefix before writing the first object (e.g. "Tal") — every custom object name starts with it.
- ☐ Reserve one contiguous ID range in app.json's idRanges before the first table exists.
- ☐ Keep Captions clean and business-readable — the prefix belongs in the object name, not the text a user reads.
- ☐ File naming (AppSource notation): Prefix_ObjectName_FullTypeName.al for a full object, ...FullTypeNameExt.al for an extension.
- ☐ Never edit a base or standard object directly — always extend it, or subscribe to a published event.
- ☐ Compile with zero errors **and** zero warnings before considering an object done.
- ☐ Ship a permission set — a full-access and a read-only pair — before go-live.

EXTENSION MODEL

If you find yourself about to open a standard object's source to change it — stop. Subscribe to a published event, or add an extension object instead. An extension that never touched a base object survives every future Business Central update automatically.